

SOAL MINI PROJECT

Format kerja: Kelompok (2 orang per kelompok) **Durasi pengerjaan:** 3 hari (8 jam per hari)
Tema aplikasi: BEBAS — kalian yang menentukan sendiri **Output:** Aplikasi web full-stack (Backend REST API + Frontend) yang berjalan end-to-end

Pendahuluan

Selamat, kalian sudah menyelesaikan seluruh rangkaian materi bootcamp — dari Git/Source Control, C# & OOP, LINQ, Async/Await, SOLID, Clean Code, EF Core, REST API, Dependency Injection, Middleware, JWT Authentication, sampai Data Binding, Directives, Pipes, Routing, Component Communication, Service, RxJS, dan Form Handling di Angular.

Final Project ini adalah kesempatan kalian membuktikan bahwa semua materi itu benar-benar bisa dirangkai jadi satu aplikasi utuh — bukan lagi latihan potongan-potongan kecil, tapi aplikasi nyata dari database sampai tampilan yang bisa diklik user.

Kalian bebas memilih tema aplikasi apa pun — toko online, perpustakaan, sistem lelang, manajemen gudang, platform donasi, atau ide kalian sendiri. Yang **tidak bebas** adalah ketentuan teknis di Section 2 di bawah ini — ketentuan ini berlaku **apa pun tema yang kalian pilih**.

Ketentuan Wajib Aplikasi

Berikut adalah 9 ketentuan yang **wajib** dipenuhi aplikasi kalian, apa pun temanya. Setiap ketentuan disertai penjelasan detail dan alasan kenapa itu diwajibkan, supaya kalian paham betul apa yang harus dibangun — bukan sekadar mengikuti instruksi tanpa mengerti tujuannya.

Ketentuan #1 — Minimal 6 Halaman (Page)

Aplikasi kalian minimal harus punya **6 halaman** yang masing-masing berdiri sendiri (punya route/URL sendiri), bukan 1 halaman raksasa yang isinya banyak section.

Contoh yang DIHITUNG sebagai halaman terpisah:

- Halaman daftar produk (`/products`)
- Halaman detail produk (`/products/:id`)
- Halaman keranjang/checkout (`/checkout`)

Contoh yang TIDAK DIHITUNG sebagai halaman terpisah:

- Modal/dialog popup di dalam 1 halaman yang sama
- Tab di dalam 1 halaman yang tidak mengubah route (kecuali memang di-desain sebagai nested route — lihat Ketentuan #7)

Ketentuan #2 — Wajib Ada Halaman Login (dengan Autentikasi Sungguhan)

Ini bukan sekadar "ada form login yang tampil di layar" — halaman Login kalian **wajib terhubung ke sistem autentikasi yang benar-benar berfungsi**. Secara spesifik, wajib ada:

1. Backend:

- Endpoint **POST /api/auth/login** yang memvalidasi email & password, lalu mengembalikan **JWT Token** kalau berhasil.
- Minimal **1 endpoint lain yang wajib [Authorize]** — artinya endpoint tersebut HARUS menolak request yang tidak membawa token valid (**401 Unauthorized**).
- Endpoint yang **[Authorize]** tersebut mengambil identitas user (**userId**) dari **klaim token JWT**, bukan dari input yang dikirim client.

2. Frontend:

- **AuthService** yang menyimpan token setelah login sukses.
- **HTTP Interceptor** yang otomatis menyisipkan header **Authorization: Bearer <token>** ke setiap request yang butuh autentikasi.
- **Route Guard (CanActivate)** yang diterapkan di minimal 1 halaman — kalau user belum login dan mencoba membuka halaman tersebut, otomatis diarahkan ke halaman Login.

Ketentuan #3 — Minimal 8-10 Tabel di Database

Aplikasi kalian wajib punya minimal **10 tabel**, dengan struktur relasi sebagai berikut (nama tabel bebas sesuai tema kalian, tapi PERAN dan RELASI di bawah ini wajib ada):

Peran Tabel	Jumlah Disarankan	Fungsi	Contoh (tema toko buku)
User	1 tabel	Data akun & autentikasi	User
Master/Referensi	2-3 tabel	Data acuan, biasanya tidak dibuat langsung oleh user, cukup ditampilkan sebagai pilihan/filter	Category, Publisher

Entity Inti	2 tabel	Objek utama aplikasi kalian	<i>Book, Author</i>
Entity Transaksional	1-2 tabel	Data hasil aksi user (yang berkaitan dengan Ketentuan #4 di bawah)	<i>Order</i>
Pivot (many-to-many)	1-2 tabel	Menghubungkan 2 tabel dengan relasi banyak-ke-banyak, biasanya membawa field tambahan seperti <i>Quantity</i>	<i>Order_Item</i>
Pendukung	1-2 tabel	Data pelengkap yang terhubung ke tabel lain	<i>Address, Review</i>

Syarat relasi wajib:

- Minimal **3 relasi one-to-many** (relasi FK biasa)
- Minimal **1 relasi many-to-many** (lewat pivot table)

Catatan penting: Tidak semua dari 10 tabel ini wajib punya halaman CRUD sendiri! Tabel Master/Referensi misalnya cukup ditampilkan sebagai dropdown atau filter di halaman lain (lihat Ketentuan #1 — 6 halaman kalian tidak perlu 1:1 dengan 10 tabel).

Ketentuan #4 — Wajib Ada Perhitungan/Transaksi Bisnis

Aplikasi wajib punya minimal 1 fitur dengan logika kalkulasi bisnis di Backend (Service layer). Perhitungan **tidak boleh berupa operasi aritmatika tunggal** seperti sekadar $\text{total} = \text{harga} \times \text{jumlah}$ atau $\text{total} = \text{subtotal} + \text{pajak}$. Perhitungan dianggap memenuhi ketentuan ini jika mengandung **minimal 2 dari 4 elemen** berikut:

1. **Logika bertingkat/kondisional** — tarif atau nilai berbeda tergantung rentang tertentu (contoh: jam pertama tarif A, jam selanjutnya tarif B; atau diskon berbeda tergantung total belanja).
2. **Turunan dari selisih/durasi** — nilai input mentah (misal waktu masuk & keluar, tanggal pinjam & kembali) harus diolah dulu menjadi durasi/selisih sebelum bisa dihitung jadi nilai uang.
3. **Pembulatan dengan aturan bisnis** — bukan pembulatan matematis biasa, tapi aturan spesifik (misal "kelebihan di atas 15 menit dibulatkan jadi 1 jam penuh").
4. **Batas nilai (cap/floor)** — ada nilai maksimum atau minimum yang membatasi hasil akhir perhitungan, walau input mentahnya melebihi itu.

Contoh yang MEMENUHI ketentuan:

- Tarif parkir: biaya dihitung dari selisih jam masuk-keluar, dengan tarif jam pertama berbeda dari jam berikutnya, dan ada tarif maksimum per hari.
- Denda keterlambatan buku perpustakaan: $\text{hari_terlambat} \times \text{tarif_per_hari}$, tapi dengan tarif berbeda untuk 3 hari pertama vs seterusnya, dan ada denda maksimum.
- Biaya pengiriman: dihitung dari berat + jarak (zona), dengan tarif berbeda per zona dan pembulatan berat ke kelipatan 0.5kg.
- Perhitungan komisi sales: persentase komisi berjenjang tergantung total penjualan (misal $<10\text{jt} = 5\%$, $10\text{-}50\text{jt} = 7\%$, $>50\text{jt} = 10\%$).
- Perhitungan pajak penghasilan (PPH) progresif: tarif pajak berjenjang tergantung besaran penghasilan kena pajak (misal $<60\text{jt} = 5\%$, $60\text{jt}\text{--}250\text{jt} = 15\%$, $>250\text{jt} = 25\%$) — beda dengan $\text{total} = \text{subtotal} + (\text{subtotal} \times \text{pajak})$ yang tarifnya tunggal/flat sehingga tidak dihitung memenuhi.

Contoh yang TIDAK memenuhi ketentuan (terlalu sederhana):

- $\text{total} = \text{harga_satuan} \times \text{jumlah}$
- $\text{total} = \text{subtotal} + (\text{subtotal} \times \text{pajak})$
- $\text{grand_total} = \text{jumlah}(\text{subtotal semua item})$ tanpa ada aturan tambahan apapun

Ketentuan #5 — Endpoint API Lengkap (Full CRUD)

Backend kalian wajib menyediakan minimal 1 endpoint untuk **setiap** jenis operasi berikut:

Method	Fungsi	Wajib Ada Minimal
GET (list)	Menampilkan daftar data	1 endpoint
GET (detail)	Menampilkan 1 data spesifik by id	1 endpoint
POST	Membuat data baru	1 endpoint
PUT	Mengubah/update data yang sudah ada	1 endpoint
DELETE	Menghapus/membatalkan data	1 endpoint

Ketentuan #6 — Wajib Ada Reactive Form

Aplikasi kalian wajib punya:

- **Minimal 1 halaman** yang formnya dibangun dengan **Reactive Form** (`FormGroup/FormBuilder`), dan form ini wajib punya **minimal 1 custom validator** (validasi yang kalian buat sendiri, bukan bawaan Angular seperti `required/email`).

Ketentuan #7 — Wajib Ada Nested/Child Route

Catatan: ini adalah pengecualian tab yang disebutkan di Ketentuan #1 — tab biasa yang tidak mengubah route TETAP TIDAK dihitung sebagai halaman terpisah, kecuali diimplementasikan sebagai nested route seperti ini.

Minimal 1 halaman kalian wajib menggunakan **Nested Route** (child route) — yaitu 1 halaman yang di dalamnya punya `<router-outlet>` KEDUA untuk menampilkan sub-halaman/tab tanpa berpindah halaman utama.

Contoh penerapan: Halaman detail 1 data yang punya beberapa tab (misal tab "Informasi Umum" dan tab "Riwayat"), di mana tiap tab sebenarnya adalah component/route terpisah yang di-render lewat router-outlet kedua tersebut.

Ketentuan #8 — Wajib Ada Exception Handling, Middleware, dan Logging

Backend kalian wajib punya:

- **Global Exception Middleware** — 1 middleware yang menangkap semua error tak terduga di seluruh aplikasi dan mengembalikan response JSON yang rapi (bukan stack trace mentah bocor ke client).
- **Minimal 1 Custom Exception** — exception buatan sendiri untuk kasus bisnis spesifik (misal: `DataNotFoundException`, `InvalidOperationException` versi kalian sendiri), yang di-mapping ke status code yang tepat (404, 400, dst).
- **Logging** — menggunakan `ILogger` (built-in .NET) minimal untuk mencatat kejadian penting (misal: saat data berhasil dibuat, saat terjadi error) ke console atau file.

Ketentuan #9 — Wajib Ada Struktur Project & Source Control yang Jelas

Struktur folder project kalian wajib mengikuti format berikut:

```
<namaAplikasi>/
├── <namaAplikasi>_BE/   (Backend)
└── <namaAplikasi>_FE/   (Frontend)
```

Contoh: kalau nama aplikasi kalian "TokoBuku", strukturnya jadi TokoBuku/TokoBuku_BE/ dan TokoBuku/TokoBuku_FE/.

Seluruh struktur project ini wajib berada di dalam **Git repository** kelompok kalian, supaya seluruh anggota bisa saling berkolaborasi (push, pull, commit bersama) — bukan dikerjakan sendiri-sendiri lalu digabung manual di akhir.

Setiap anggota kelompok wajib berperan di FE maupun BE — tidak boleh pembagian kerja "1 orang pegang FE saja, 1 orang pegang BE saja". Riwayat commit harus mencerminkan kontribusi kedua sisi dari masing-masing anggota.

Pengecualian: khusus untuk **migrasi database (DB migration)**, tetap boleh dikerjakan oleh 1 orang saja.

3. Batasan Penting

- **Tema bebas, tapi 9 ketentuan di atas berlaku wajib** — apapun tema yang kalian pilih, pastikan tema tersebut memungkinkan semua ketentuan di atas terpenuhi secara natural (khususnya Ketentuan #3 soal relasi tabel dan Ketentuan #4 soal perhitungan bisnis).
- **Semua ketentuan ini disusun hanya dari materi yang sudah diajarkan** selama bootcamp — tidak akan ada tuntutan menggunakan library, teknik, atau konsep di luar itu.
- **6 halaman ≠ 10 tabel satu-satu** — seperti dijelaskan di Ketentuan #3, tidak semua tabel butuh halaman sendiri. Fokuskan 6 halaman kalian ke alur yang paling penting dari aplikasi kalian.